

PPL Lecture-5

Introduction and Elements of C Language

Overview of C

- C is developed by Dennis Ritchie
- C is a structured programming language
- C supports functions that enables easy maintainability of code, by breaking large file into smaller modules
- Comments in C provides easy readability
- C is a powerful language

Character set of C

- Source character set
 - Alphabets
 - Digits
 - Special character
 - White space

Alphabets and Digits

- Alphabets are divided into two group in C language.
 - Uppercase letter: A, B, ..., Z
 - Lowercase letter: a,b, ..., z
- Digits
 - 0, 1, ..., 9

Special characters

Special character	Character represented
!	Exclamation mark
"	Quotation mark
#	Number sign
\$	Dollar Sign
%	Percent sign
&	Ampersand
(	Left parenthesis
)	Right parenthesis
*	Asterisk
+	Plus sign
,	Comma
-	Mins sign
.	Period
/	Slash
:	Colon
;	Semicolon
<	less then sign
=	Equal sign

Whitespace character

Constant	Character represented
<code>\b</code>	Blank space
<code>\\</code>	Back slash
<code>\n</code>	New line
<code>\f</code>	Form feed
<code>\0</code>	Null
<code>\v</code>	Vertical tab
<code>\t</code>	Horizontal tab
<code>\'</code>	Single quote
<code>\r</code>	Carriage return
<code>\a</code>	Alarm(bell)
<code>\?</code>	Question mark

Keywords/Reserved Words in C

- Keywords are predefined, reserved words used in programming that have special meanings to the compiler
- 32 reserved words
- always written in lowercase

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Identifiers

- Identifier refers to name given to entities such as variables, functions, structures etc.
- **Rules for naming identifiers**
 - A valid identifier can have letters (both uppercase and lowercase letters), digits and underscores.
 - The first letter of an identifier should be either a letter or an underscore.
 - You cannot use keywords like int, while etc. as identifiers.
 - There is no rule on how long an identifier can be. However, you may run into problems in some compilers if the identifier is longer than 31 characters.

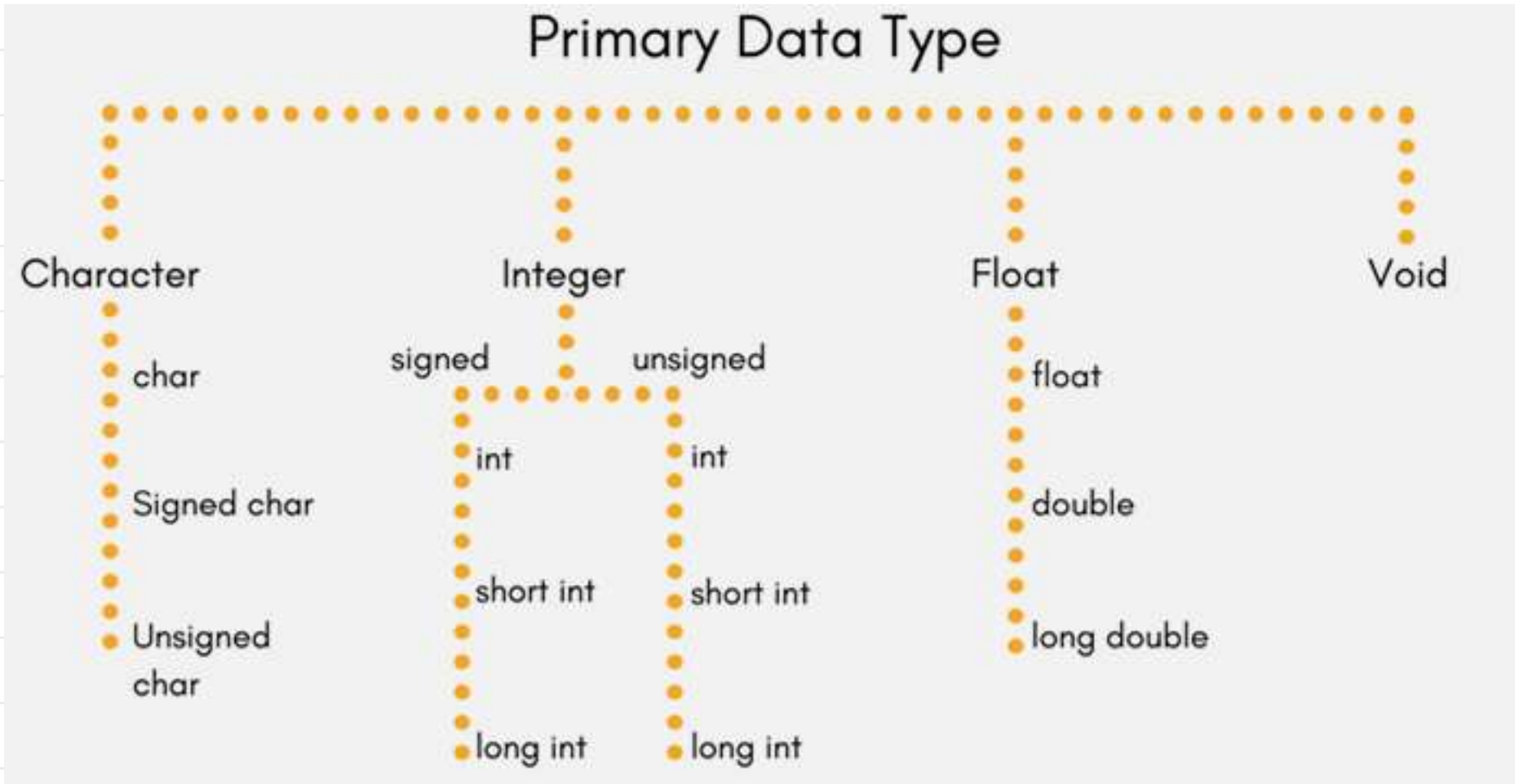
Constants

- Constant is a value that cannot be changed during execution of the program.
- Types:
 - Integer constant: 0, 123, -3705
 - Real (floating point) constant: 0.5, 5.3, -0.0073
 - Character constant: '9', 'D', '\$', ' ', '#'
 - String constant: "Kumar", "593", "8", " ", "A"

Data types in C

- Primary data types
 - int, float, double, char
- Aggregate data types
 - Arrays come under this category
 - Arrays can contain collection of int or float or char or double data
- User defined data types
 - Structures and enum fall under this category.

Primary Data Types



Integer type

- Integers are used to store whole numbers.

Type	Size(bytes)	Range
int	2	-32,768 to 32767
unsigned int	2	0 to 65535
long int	4	-2,147,483,648 to 2,147,483,647

Floating point type

- Floating point type are used to store fractional numbers (real numbers).

Type	Size(bytes)	Range	Precision
float	4	3.4E-38 to 3.4E+38	6 decimal places
double	8	1.7E-308 to 1.7E+308	15 decimal places
long double	10	3.4E-4932 to 1.1E+4932	19 decimal places

Character type

- Character types are used to store characters value.

Type	Size(bytes)	Range
char	1	-128 to 127
unsigned char	1	0 to 255

Program structure

- A sample C Program:

```
#include<stdio.h>
int main()
{
    --other statements
    // Comments after double slash
}
```

Header files

- The files that are specified in the include section is called as header file
- These are precompiled files that has some functions defined in them
- We can call those functions in our program by supplying parameters
- Header file is given an extension .h
- C Source file is given an extension .c

Main function

- This is the entry point of a program
- When a file is executed, the start point is the main function
- From main function the flow goes as per the programmers choice.
- There may or may not be other functions written by user in a program
- Main function is compulsory for any C program

Writing the first program

```
#include<stdio.h>
int main()
{
    printf("Hello World");
    return 0;
}
```

- This program prints **Hello World** on the screen when we execute it

Running a C Program

- Type a program
- Save it
- Compile the program – This will generate an exe file (executable)
- Run the program (Actually the exe created out of compilation will run and not the .c file)
- In different compiler we have different option for compiling and running. We give only the concepts.

Comments in C

- Generally every programming language have comments to maintain proper explanation about the code.
- It is always a good idea to write comments for the code to understand the logic behind the statements.
- **Single line comment**
 - // (double slash)
 - Termination of comment is by pressing enter key
- **Multi line comment**

```
/* ....  
..... */
```

This can span over to multiple lines.

Variables

- Variable is a name that can be used to store a value.
- There values can be changed during execution of the program.
- **Declaration** of a variable: *datatype variable_name;*
 - `int x;`
 - `float salary;`
 - `char grade;`
- **Initialization** of a variable: *variable_name = initial_value;*
 - `int a = 5;`
 - `float x = 8.9, y = 3.5;`

Example: Variable Declaration

- The following is an example of some variable names:

average

pi

number_of_students

- The following are *not* variable names:

3rd_entry

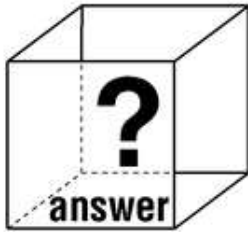
all\$done

the end

int

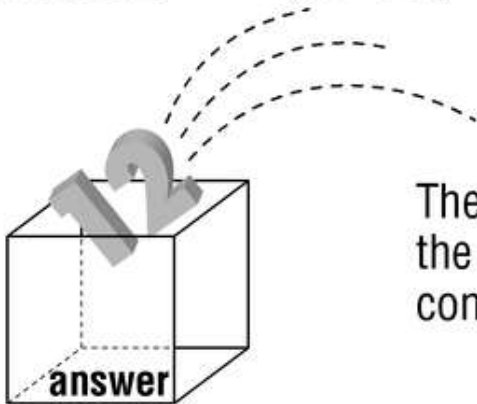
Example: Variable Initialization

A `int answer;`



The variable `answer` has not been assigned a value. So we put a “?” in it to indicate that it’s in an unknown state.

B `answer = (1+2) * 4;`



The variable `answer` is assigned the value of the expression $(1+2) * 4$. The box is shown containing the value 12.

Input-Output in C

- To use any function from the standard I/O library, we have to include **stdio.h**.
 - **Input function:** scanf(), getchar()
 - **Output functions:** printf(), putchar()
- scanf() and printf() make use of format specifier to specify the type and size of data.

Format specifier	Type
%c	A single character
%d	A decimal integer
%f	A floating point number
%lf	Long range of floating point number (double)
%x	A hexadecimal integer
%s	A string

Reading Input Data

- **Syntax:** `scanf ( “format specifier” , &var1, &var2, ... ) ;`
- **Examples:**

```
char ch;
```

```
scanf(“%c” , &ch);
```

```
int marks;
```

```
scanf(“%d”, &marks);
```

```
float salary;
```

```
scanf(“%f”, &salary);
```

Writing Output Data

- **Syntax:** printf (“format specifier”, var1, var2, ...) ;

- **Examples:**

```
char ch;
```

```
scanf(“%c” , &ch);
```

```
printf(“%c”, ch);
```

```
int marks = 75;
```

```
printf(“%d”, marks);
```

```
float height, width;
```

```
scanf(“%f %f”, &height, &width);
```

```
printf(“%f”, height*width);
```

Operators in C

- **Operators** are symbols which take one or more **operands** or **expressions** and perform arithmetic or logical computations.
- Types of operators:
 - Arithmetic Operators
 - Relational Operators
 - Logical Operators
 - Bitwise Operators
 - Assignment Operators
 - Misc Operators

Arithmetic Operators

- Arithmetic operators are used for numeric calculations. They are of two types -
 1. Unary arithmetic operator
 2. Binary arithmetic operators
- **Unary Arithmetic Operators**- They require only one operand.
 - Ex: $+x$, $-y$, $+78$, 23 , -9 , 0.012 , -20.87
- **Binary Arithmetic Operators**- They require two operands.
 - $+$, $-$, $*$, $/$, $\%$

Assignment Operators

- This assignment operator "=", is used in assignment expressions and assignment statements.
- The operand on the LHS should be a variable, while the operand on the RHS can any variable, constant or expression.
- Examples:
 - $x = 8$
 - $y = 5$
 - $s = x + y - 2$
 - $y = x$
 - $a = b = c = 20$
 - $x = x + 5$
 - $y = y / 10$

Increment And Decrement Operators

- Increment operator: ++
 - increments the value of the variable by 1
- Decrement operator: --
 - decrements the value of the variable by 1
- Two types:
 1. **Prefix increment / decrement** - operator is written before the operand (e.g. ++x or --x)
 2. **Postfix increment / decrement** - operator is written after the operand (e.g. x++ or x--)

Relational Operators

- They are used to compare values of two expressions depending on their relations.
 - If the relation is *true* then the value of relational expression is 1 .
 - If the relation is *false* then the value of expression is 0.

Operator	Meaning
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	equal to
!=	not equal to

Logical or Boolean Operators

- An expression that combines two or more expressions is termed as a logical expression.
- For combining these expressions we use logical operators.
- These operators return 0 for false and 1 for true.

Operator	Meaning
&&	AND
	OR
!	NOT

Precedence and Associativity of Operators

- **Operator precedence** determines which operator is performed first in an expression with more than one operators with different precedence.
- **Example:** $10 + 20 * 30$ is calculated as $10 + (20 * 30)$ and not as $(10 + 20) * 30$
- **Operators Associativity** is used when two operators of same precedence appear in an expression. Associativity can be either **Left to Right** or **Right to Left**.
- **Example:** ‘*’ and ‘/’ have same precedence and their associativity is **Left to Right**, so the expression “ $100 / 10 * 10$ ” is treated as “ $(100 / 10) * 10$ ”.

Precedence and Associativity of Operators

Category	Operator	Associativity
Postfix	++ --	Left to right
Unary	+ - ! ~ ++ --	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=	Right to left